COMP4616 – Fundamentals of Machine Learning

ASST. PROF. TUĞBA ERKOÇ

SPRING 2026

WEIGHT INITIALIZATION, BATCH NORMALIZATION, REGULARIZATION TECHNIQUES, TRAINING NEURAL NETWORKS

Weight Initialization

Why Weight Initialization Matters

Too Small

- Activations vanish layer by layer.
- Gradients become zero and learning stops
- The vanishing gradient problem.

Too Large

Activations explode, gradients overflow, and the network becomes numerically unstable during training.

Just Right

Properly scaled weights keep activations stable across all layers, enabling fast and reliable convergence.

The right initializer depends on your activation function and network depth.

Weight Initialization

- Proper weight initialization helps ensure that the model converges effectively and achieves high accuracy.
- When weights are not initialized correctly, it can lead to issues during training like vanishing or exploding gradients.
  - If weights are too small → activations may vanish, resulting in ineffective learning.
  - If weights are too large → activations may explode, causing instability.
- Properly initialized weights provide a good starting point for optimization, allowing the model to learn effectively.
- There are a couple of weight initialization techniques:
  - Zero initialization
  - Random initialization
  - Xavier (Glorot) initialization
  - He Initialization
- Choosing the right weight initialization technique depends on the specific architecture, activation functions, and problem you're working on.
- Experimenting with different methods can lead to better performance.

Four Weight Initialization Techniques

Zero

✖ Avoid

All weights = 0. Every neuron computes the same gradient; no diversity, no learning. A dead end.

Random

⚠ Shallow only

Breaks symmetry but ignores layer size. Works for small nets; causes instability in deep networks.

Xavier / Glorot

✓ Sigmoid, Tanh

Var = 2 / (fan_in + fan_out). Keeps signals balanced across layers for saturating activations.

He

✓ ReLU family

Var = 2 / fan_in. Doubles Xavier's variance to compensate for ReLU zeroing negative inputs.

Initializer - Activation Pairing

Method	Formula (Variance)	Use With	Why It Works
Zero	`w = 0`	Nothing	Symmetric neurons → identical gradients. Zero learning.
Random	`Fixed small σ (e.g. 0.01)`	Shallow nets	Breaks symmetry; ignores scale → instability in deep nets.
Xavier / Glorot	`Var = 2/(fan_in + fan_out)`	Sigmoid, Tanh	Balances input/output fan to keep activations stable.
He	`Var = 2/fan_in`	ReLU, Leaky ReLU	ReLU kills ~50% of activations; He doubles variance to compensate.

Keras: Glorot Initialization

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),

    keras.layers.Dense(25,
        activation='tanh',
        kernel_initializer='glorot_uniform',
        bias_initializer='zeros'),

    keras.layers.Dense(10,
        activation='sigmoid',
        kernel_initializer='glorot_uniform',
        bias_initializer='zeros')
])
```

glorot_uniform

Xavier initialization is ideal for tanh and sigmoid layers.

'zeros' bias

Bias starts at zero; only weights need careful initialization.

Tip

Glorot is Keras default for Dense layers.
He init: use 'he_normal'.

Batch Normalization

The Problem: Unscaled Features

Example: Predicting Car Prices

Age → 0 – 70

Small range

Kilometers → 0 – 500,000

Huge range

Gradient imbalance

SGD shifts weights proportional to activation size.
Large scale features dominate.

Unbalanced weights

Some weights grow very large, others stay tiny
The distribution becomes chaotic.

Unstable training

Loss oscillates or diverges
The network cannot reliably converge

Fix: normalize inputs before training, and apply Batch Normalization during training.

How Batch Normalization Works

Internal Covariate Shift: The distribution of layer inputs shifts every minibatch as weights update

1

Compute μ and σ

Calculate mean and standard deviation of activations within the current minibatch.

2

Normalize

Subtract μ and divide by σ to give activations mean = 0, variance = 1.

3

Scale & Shift

Multiply by γ and add β, learnable parameters letting the network undo normalization if needed.

4

Stable Distribution

Each layer now receives well conditioned inputs regardless of upstream weight changes.

Keras: Two Placement Styles

Style A — BN before Activation (original paper)

```
keras.layers.Dense(64),
keras.layers.BatchNormalization(),
keras.layers.Activation('relu'),
```

Style B — BN after Activation (modern practice)

```
keras.layers.Dense(64,
activation='relu'),
keras.layers.BatchNormalization(),
```

Stabilizes Training

Reduces internal covariate shift so each layer learns from a consistent input distribution.

Faster Convergence

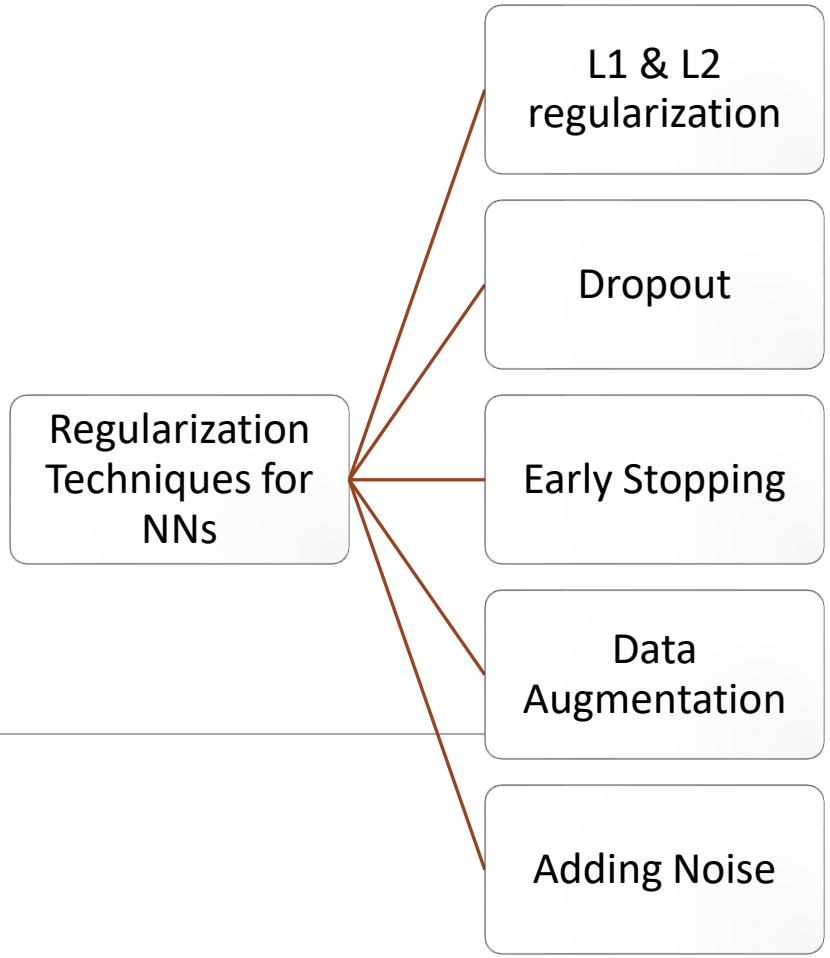
Networks with BN often need far fewer epochs to reach comparable accuracy.

Regularization Effect

The noise introduced by mini-batch statistics acts like a mild regularizer, reducing overfitting.

Both styles are valid. Style B is more common in modern implementations. Check your framework's defaults.

Regularization Techniques



Overfitting: The Core Problem

Regularization = any technique that reduces generalization error without reducing training error.

When Overfitting Happens

Model memorizes training data including noise

Training loss → 0, validation loss → rises

Too many parameters for the amount of data

No mechanism to control model complexity

Regularization Techniques

L1 & L2

Penalty terms on weight size

Dropout

Randomly disable neurons

Early Stopping

Stop before the model memorizes

Data Augmentation

Expand training distribution

L1 & L2 Regularization

L1 — Lasso

Loss += λ · Σ|w|

- Drives many weights to exactly zero.
- Acts as automatic feature selection; irrelevant features get zeroed out. Leads to a simpler model
- Best for: high dimensional data with many irrelevant features.

L2 — Ridge

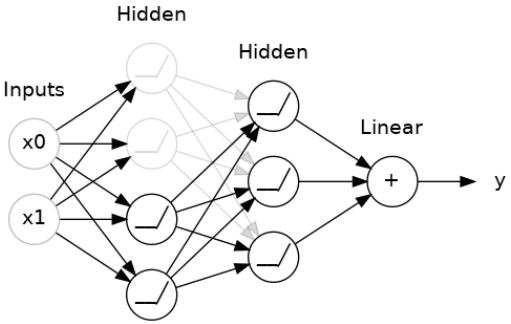
Loss += λ · Σw²

- Shrinks all weights toward zero without eliminating any, reducing their impact
- Penalizes large coefficients, promoting a smoother model.
- Best for: general overfitting in deep networks.

Keras

```
from keras.regularizers import L1, L2, L1L2

kernel_regularizer=L1(0.01)    # L1 only
kernel_regularizer=L2(0.01)    # L2 only
kernel_regularizer=L1L2(l1=0.01, l2=0.01)  # Combined
```

Dropout



Each training step

Randomly **zero out** a fraction **p** of neuron activations

Those neurons don't contribute to forward or backward pass

Different neurons drop every iteration → ensemble effect

At test time: Dropout is OFF. All neurons active

Dropout Rate Guide

`0.1–0.3`
Input layers or light regularization

`0.4–0.5`
Hidden layers, most common range

`> 0.5`
Too aggressive, destroys information

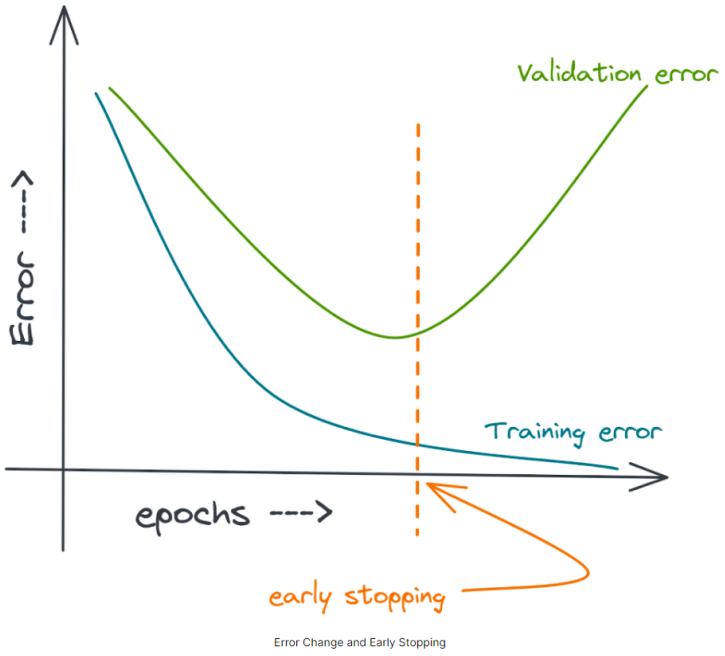
```
keras.layers.Dropout(0.5)   # 50% dropout

# In a model:
keras.layers.Dense(128, activation='relu'),
keras.layers.Dropout(0.4),
keras.layers.Dense(10, activation='softmax')
```

Early Stopping

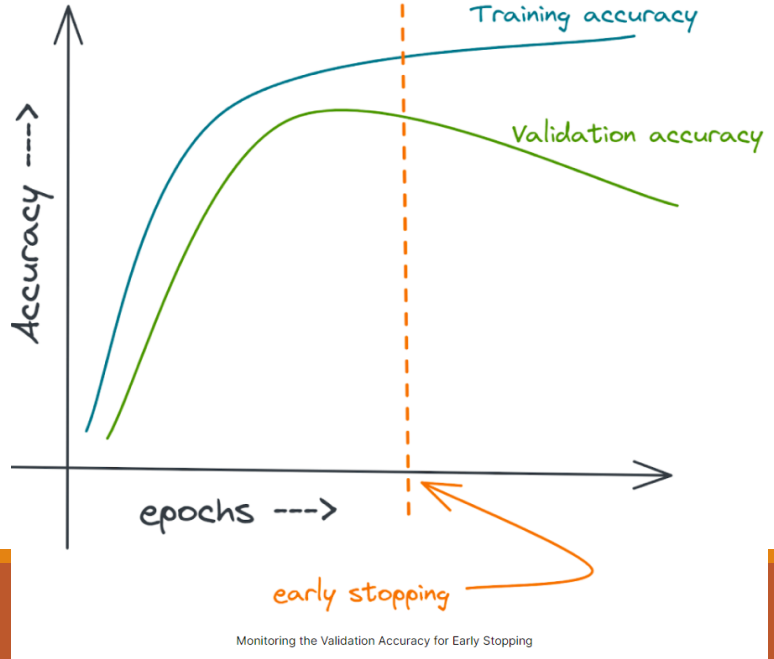
Early Stopping

- Monitor validation loss each epoch.
- Stop training when it stops improving.
- Restore the best checkpoint.

Keras
EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)



Error Change and Early Stopping

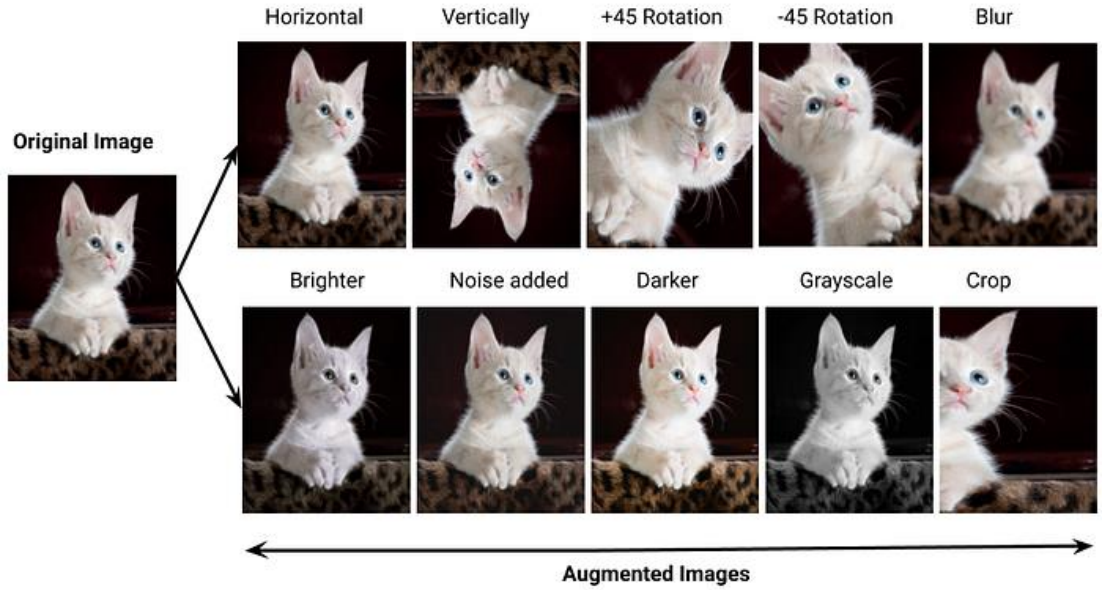


Monitoring the Validation Accuracy for Early Stopping

Data Augmentation

Data Augmentation

- Create new training examples by applying label preserving transforms
  - Flip
  - Rotate
  - Crop
  - Add noise
  - Shift
  - Zoom
- Especially useful when training data is limited.



Early Stopping & Data Augmentation

When to Use Which

Small dataset

Dropout + Data Augmentation

Many irrelevant features

L1 (feature selection)

General deep net overfitting

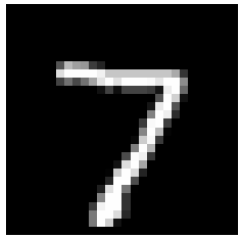
L2 + Dropout

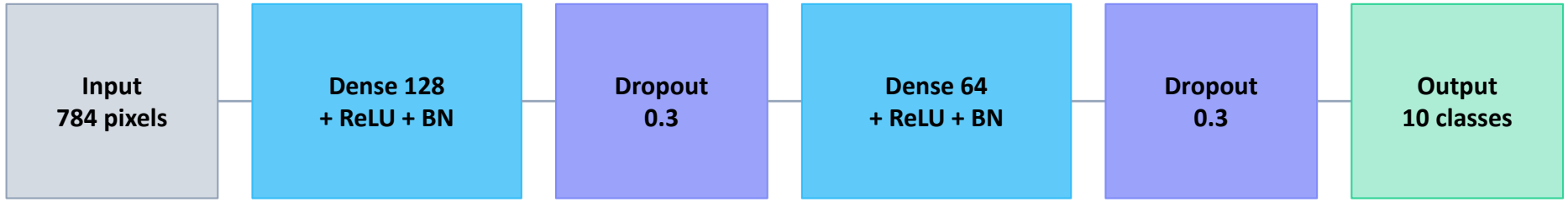
Unsure when to stop

Early Stopping

Tip: combining L2 + Dropout + Early Stopping is standard practice.

Training an MLP on MNIST





MNIST Dataset

- 60,000 training images
- 28×28 pixels
- 10 classes (digits 0–9)
- Grayscale

```
model = keras.Sequential([
  keras.layers.Flatten(input_shape=(28,28)),
  keras.layers.Dense(128, activation='relu',
      kernel_initializer='he_normal'),
  keras.layers.BatchNormalization(),
  keras.layers.Dropout(0.3),
  keras.layers.Dense(64, activation='relu',
      kernel_initializer='he_normal'),
  keras.layers.BatchNormalization(),
  keras.layers.Dropout(0.3),
  keras.layers.Dense(10, activation='softmax')
])
```

https://adamharley.com/nn_vis/mlp/3d.html